CANTIDADES Y REPORTE BÁSICO

VALERIA MONCADA GONZÁLEZ

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO

LÓGICA DE PROGRAMACIÓN G103

EDWIN FERNEY BARRAGÁN MUÑOZ

MEDELLÍN – ANTIOQUIA

19/04/2026

En este primer paso se declaran las variables.

```
// Paso 1: Estructura de datos
struct Producto {
    string nombre;
    string codigo;
    int cantidad;
    double precio;
};
```

```
const int MAX_PRODUCTOS = 100;
Producto inventario[MAX_PRODUCTOS];
int totalProductos = 0;
```

MAX_PRODUCTOS: Es la constante que define el tamaño máximo del arreglo (100 productos). Al ser constante, su valor no puede modificarse durante la ejecución.

inventario[MAX_PRODUCTOS]: es el arreglo estático de tipo **Producto**. Cada posición del arreglo almacena un objeto con los cuatro atributos definidos en la estructura.

totalProductos: es la variable global que lleva el conteo de cuantos productos han sido cargados en el arreglo. Se inicia en 0.

```
// Paso 2: Función para actualizar cantidad
void actualizarCantidad() {
    string codigoBuscado;
    int nuevaCantidad;

    cout << "\n--- ACTUALIZAR CANTIDAD ---" << endl;
    cout << "Ingrese el codigo del producto: ";
    cin >> codigoBuscado;

    bool encontrado = false;
    for (int i = 0; i < totalProductos; i++) {
        if (inventario[i].codigo == codigoBuscado) {
            cout << "Producto encontrado: " << inventario[i].nombre << endl;
            cout << "Cantidad actual: " << inventario[i].cantidad << endl;
            cout << "Ingrese la nueva cantidad: ";
            cin >> nuevaCantidad;
            inventario[i].cantidad = nuevaCantidad;
            cout << "Cantidad actualizada correctamente." << endl;
            encontrado = true;
            break;
        }
    }

    if (!encontrado) {
        cout << "Producto con codigo '" << codigoBuscado << "' no encontrado." << endl;
    }
}
```

void actualizarCantidad(): es una función que no retorna ningún valor. Se llama desde el menú cuando el usuario elige la opción 1.

string codigoBuscado / int nuevaCantidad: son variables locales que almacenan temporalmente el código ingresado por el usuario y la nueva cantidad.

cin >> codigoBuscado: lee el código que el usuario ingresa.

bool encontrado = false: bandera booleana que controla si el producto fue hallado o no. Inicia en false.

for (int i = 0; i < totalProductos; i++): es un ciclo que recorre cada posición del arreglo desde 0 hasta el total de productos registrados.

if (inventario[i].codigo == codigoBuscado): esto compara el código de cada producto con el código ingresado por el usuario.

inventario[i].cantidad = nuevaCantidad: si hay coincidencia, sustituye la cantidad del producto con el nuevo valor.

break: interrumpe el ciclo evitando acciones innecesarias.

if (!encontrado): si al terminar el ciclo la bandera sigue en false, informa al usuario que el código no existe en el inventario.

```
// Paso 3: Función para generar reporte de bajo inventario
void generarReporte() {
    cout << "\n--- REPORTE DE BAJO INVENTARIO (menos de 5 unidades) ---" << endl;
    bool hayProductos = false;

    for (int i = 0; i < totalProductos; i++) {
        if (inventario[i].cantidad < 5) {
            cout << "Nombre:   " << inventario[i].nombre << endl;
            cout << "Codigo:   " << inventario[i].codigo << endl;
            cout << "Cantidad: " << inventario[i].cantidad << endl;
            cout << "Precio:   $" << inventario[i].precio << endl;
            cout << "----------------------------" << endl;
            hayProductos = true;
        }
    }

    if (!hayProductos) {
        cout << "No hay productos con bajo inventario." << endl;
    }
}
```

bool hayProductos = false: sirve para detectar si existe algún producto con bajo inventario. Si al final sigue en false, significa que todos los productos tienen 5 o más unidades.

for (int i = 0; i < totalProductos; i++): ciclo que recorre todos los productos del arreglo.

if (inventario[i].cantidad < 5): condición que filtra únicamente los productos con cantidad menor a 5 unidades.

cout << inventario[i].nombre: muestra en la consola cada atributo del producto (nombre, código, cantidad y precio) de forma legible.

if (!hayProductos): si ningún producto cumple la condición, se muestra un mensaje indicando que no hay productos con bajo inventario.

```
void cargarProductosEjemplo() {
inventario[0] = {"Audifonos Bluetooth", "P001", 3, 59.99};
inventario[1] = {"Cable USB-C", "P002", 10, 9.99};
inventario[2] = {"Mouse Inalambrico", "P003", 2, 29.99};
inventario[3] = {"Teclado Mecanico", "P004", 7, 89.99};
inventario[4] = {"Memoria USB 64GB", "P005", 1, 14.99};
totalProductos = 5;
}
```

Esta función carga 5 productos de ejemplo directamente en el arreglo para poder probar el programa sin necesidad de ingresar datos manualmente. Usé la sintaxis de inicialización de estructuras con llaves **{}**, asignando valores en el mismo orden en que fueron declarados los atributos: nombre, código, cantidad y precio. Finalmente, actualiza **totalProductos = 5** para que los ciclos sepan cuantos productos hay.

```
int main() {
    cargarProductosEjemplo();

    int opcion;
    do {
        cout << "1. Actualizar cantidad" << endl;
        cout << "2. Generar reporte" << endl;
        cout << "0. Salir" << endl;
        cin >> opcion;

        switch (opcion) {
            case 1: actualizarCantidad(); break;
            case 2: generarReporte(); break;
            case 0: cout << "Saliendo..."; break;
            default: cout << "Opcion no valida"; break;
}

    } while (opcion != 0);
    return 0;
}
```

int main(): Todo programa en C++ comienza su ejecución aquí.

cargarProductosEjemplo(): se llama primero para tener datos listos antes de mostrar el menú.

do { ... } while (opcion != 0): ciclo do-while que garantiza que el menú se muestre al menos una vez. El ciclo se repite hasta que el usuario ingrese 0 para salir.

switch (opcion): estructura de selección múltiple que ejecuta la función correspondiente según la opción elegida por el usuario.

case 2: llama a **generarReporte()**.

case 0: muestra mensaje de despedida y termina el ciclo.

default: captura cualquier opción invalida e informa al usuario.

return 0: indica al sistema operativo que el programa termino correctamente.

MENÚ PRINCIPAL:

```
1. Actualizar cantidad
2. Generar reporte
0. Salir
```

OPCIÓN 1:

```
--- ACTUALIZAR CANTIDAD ---
Ingrese el codigo del producto: P003
Producto encontrado: Mouse Inalambrico
Cantidad actual: 2
Ingrese la nueva cantidad: 10
Cantidad actualizada correctamente.
```

OPCIÓN 2:

```
--- REPORTE DE BAJO INVENTARIO (menos de 5 unidades) ---
Nombre:   Audifonos Bluetooth
Codigo:   P001
Cantidad: 3
Precio:   $59.99
----------------------------
Nombre:   Memoria USB 64GB
Codigo:   P005
Cantidad: 1
Precio:   $14.99
----------------------------
```